Assignment 4: SDN

Software Defined Networking using Mininet, Open vSwitch and Ryu

Cloud Computing Lab (CSE12045)

Name: Deep Shekhar Halder
Roll Number: UG/02/BTCSE/2023/063
Program: B.Tech CSE, Adamas University
Instructor: Anusuya Bera

Aim

To create a Software Defined Network using Mininet, Open vSwitch and the Ryu controller, and to demonstrate basic forwarding and traffic filtering using OpenFlow flow rules.

Objectives

1. Install Mininet, Open vSwitch and the Ryu controller.
2. Build a virtual network of one switch and three hosts that is controlled by a remote controller.
3. Observe the flow rules the controller installs in the switch.
4. Add a manual flow rule that blocks the traffic of one host, then remove it.

Environment

Ubuntu running in a VirtualBox virtual machine (Ubuntu-CloudLab, 4 GB RAM, 2 CPUs), Mininet 2.3.0, Open vSwitch 3.7.1, Ryu 4.34 on Python 3.9, OpenFlow 1.3.

Result

The SDN environment was successfully created using Mininet, Open vSwitch, and the Ryu Controller. Basic forwarding through the controller was demonstrated, and a priority-100 flow rule dropped the traffic entering port 1 (h1), while h2 and h3 could still communicate.

Theory

What is SDN? Software Defined Networking separates the control plane (the logic that decides where traffic goes) from the data plane (the devices that forward packets). In a traditional network every switch makes its own decisions. In SDN a central controller makes the decisions and programs the switches, which gives centralized control, easier management and the ability to change network behaviour through software.

Architecture. An SDN has three layers: the application layer (programs such as firewalls or load balancers), the control layer (the controller, here Ryu) and the infrastructure layer (switches, here Open vSwitch). The controller talks to the switches through a southbound protocol, OpenFlow, and to applications through northbound APIs.

OpenFlow. OpenFlow is the protocol that lets the controller add, change and delete entries in a switch's flow table. Each entry has match fields (for example input port or MAC address), a priority, and actions (for example output to a port, or drop). When a packet arrives, the switch uses the matching entry with the highest priority. If nothing matches, the table-miss entry usually sends the packet to the controller.

Tool	Role in this lab
Mininet	Creates virtual hosts, switches and links on a single machine, so a whole network can be tested without hardware.
Open vSwitch (OVS)	A software switch that supports OpenFlow. It is the data plane (s1) in this experiment.
Ryu	A Python SDN controller. The simple_switch_13 application learns which MAC address is on which port and installs forwarding rules.
ovs-ofctl	A command line tool to view and edit the flow table of an OpenFlow switch.

How the learning switch works. At first the switch has no forwarding rules, so the first packet of a conversation goes to Ryu. Ryu records the source MAC address and port, floods the packet if the destination is unknown, and installs a flow rule once it knows both ends. Later packets are forwarded directly by the switch without involving the controller.

Filtering with a flow rule. A rule with a higher priority than the learned rules and the action drop stops matching traffic at the switch. Here the rule matches in_port=1 (h1), so everything h1 sends is discarded while the other hosts are not affected.

Viva Questions

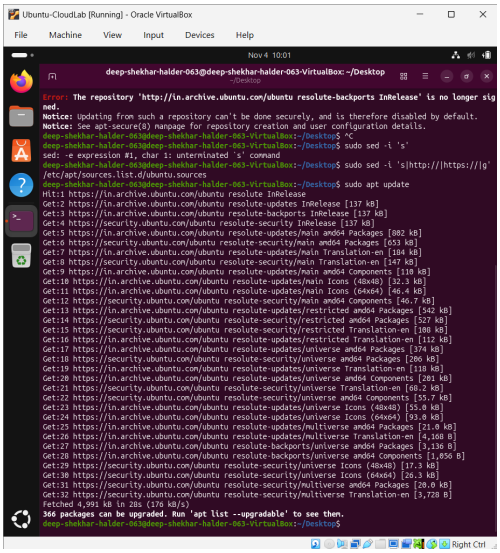
1. What is the difference between the control plane and the data plane? The control plane decides how packets are forwarded (Ryu), and the data plane forwards them (Open vSwitch).
2. What is OpenFlow? The protocol used by the controller to manage the flow table of a switch.
3. Why did the drop rule win over the learned rules? It has priority 100, and the learned rules have priority 1, so the switch applies it first.
4. What does the table-miss entry do? It sends packets that match no rule to the controller.
5. Why is Mininet used? It lets a complete network be emulated on one computer for testing.

Step 1: Update Ubuntu

```
sudo apt update && sudo apt upgrade -y
```

Explanation: Refreshes the list of available packages so the latest versions of Mininet and Open vSwitch can be found.

Observation: The package index downloaded without errors. On this network the default HTTP sources returned 403 Forbidden, so the apt sources were switched to HTTPS (sudo sed -i 's|http://|https://|g' /etc/apt/sources.list.d/ubuntu.sources) and the update then completed.

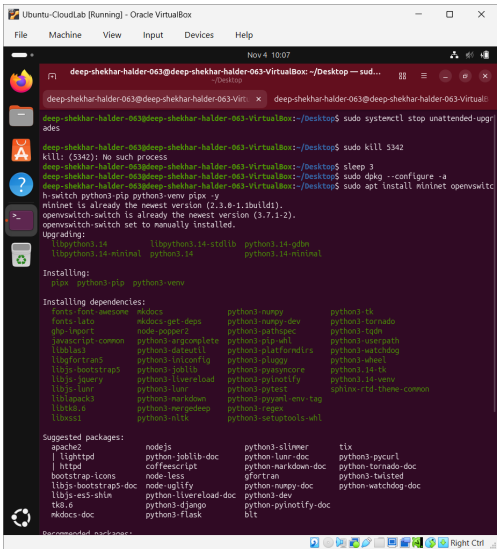


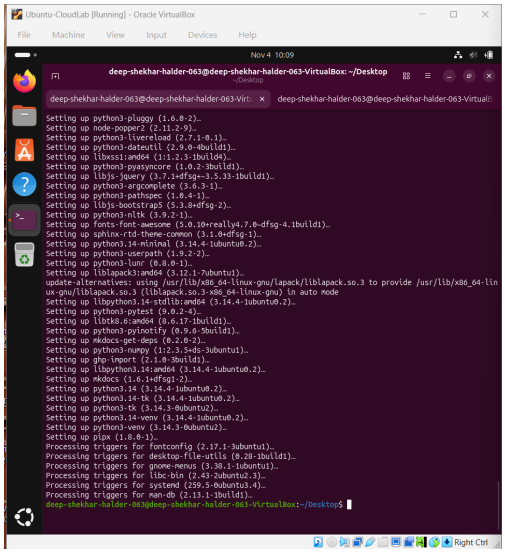
Steps 2, 3 and 6: Install Mininet, Open vSwitch, Python pip and venv

```
sudo apt install mininet openvswitch-switch python3-pip python3-venv pipx -y
```

Explanation: Mininet creates a virtual network of hosts and switches on one machine. Open vSwitch (OVS) is a software switch that supports OpenFlow, so it acts as the SDN data plane. python3-pip and python3-venv are needed to install Ryu in an isolated Python environment.

Observation: The packages were fetched and installed (Mininet 2.3.0 and Open vSwitch 3.7.1). Ubuntu's automatic updater was holding the package lock for a while, so the install started only after it finished.



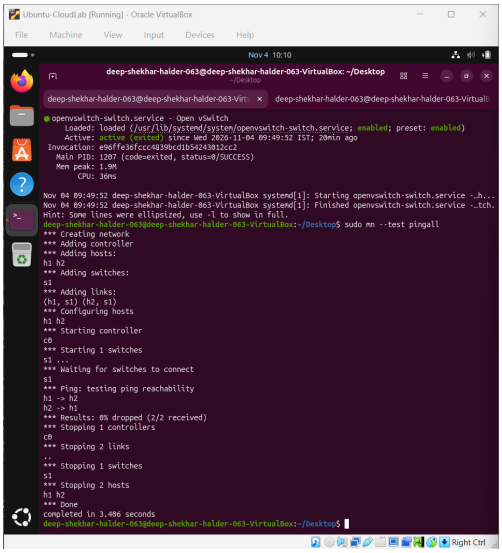


Steps 4 and 5: Check Open vSwitch and test Mininet

```
sudo systemctl status openvswitch-switch
sudo mn --test pingall
```

Explanation: The first command shows whether the Open vSwitch service is enabled. The second builds a default Mininet network (h1, h2, s1), pings between all hosts and exits.

Observation: The service is enabled and shows active (exited), which is normal because it only starts the OVS background daemons. The test network reported 0% dropped (2/2 received), so Mininet and OVS work together.

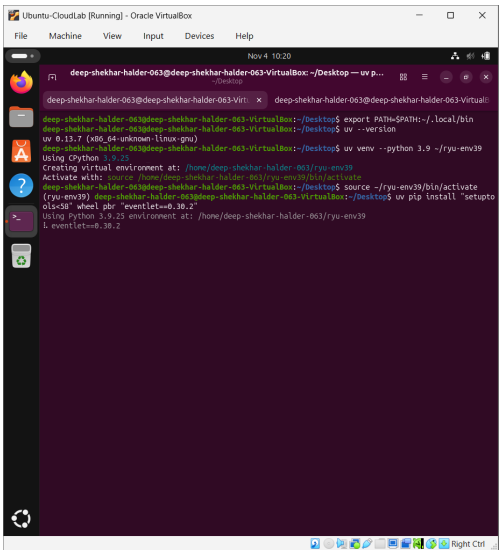


Step 7: Create and activate the Ryu environment

```
uv venv --python 3.9 ~/ryu-env39
source ~/ryu-env39/bin/activate
```

Explanation: A virtual environment keeps Ryu's libraries separate from the system Python. Ryu 4.34 does not build on the very new default Python of this Ubuntu, so a Python 3.9 environment was created with the uv tool.

Observation: uv downloaded CPython 3.9.25 and created the environment. The prompt starts with (ryu-env39) once it is active.

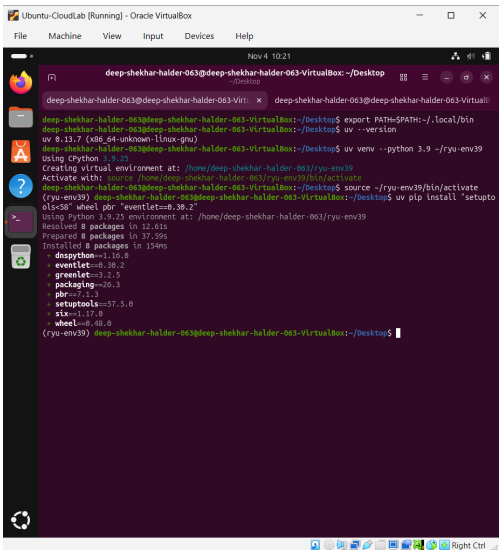


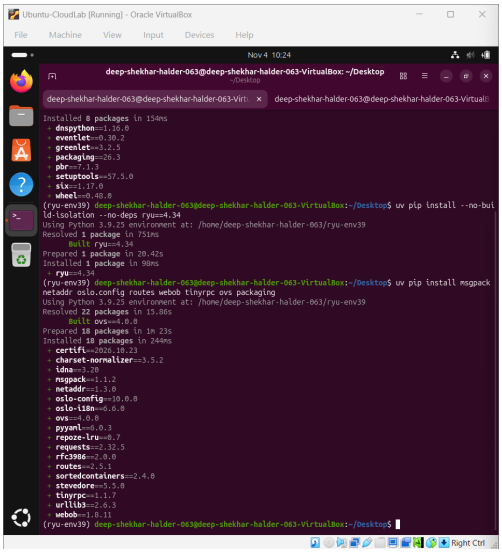
Step 8: Install Ryu

```
uv pip install "setuptools<58" wheel pbr "eventlet==0.30.2"
uv pip install --no-build-isolation --no-deps ryu==4.34
uv pip install msgpack netaddr oslo.config routes webob tinyrpc ovs packaging
```

Explanation: Ryu is a Python-based SDN controller. Its build script needs an older setuptools and a compatible eventlet, so these were installed first and Ryu was then installed without build isolation. The last command adds the libraries Ryu imports at run time.

Observation: All packages installed successfully, including ryu==4.34.



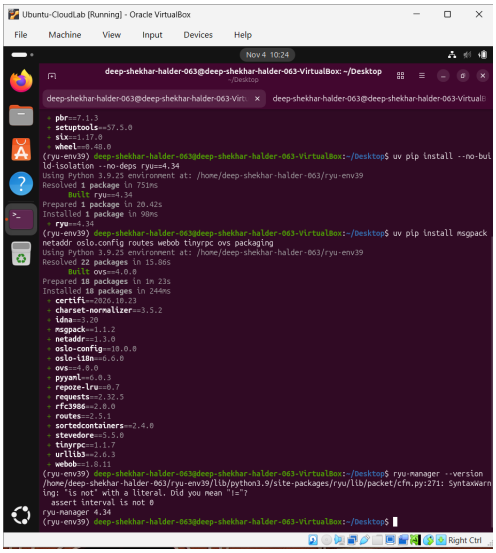


Step 9: Test the Ryu installation

```
ryu-manager --version
```

Explanation: Prints the installed version of the Ryu manager, which confirms that the controller can start.

Observation: The output is ryu-manager 4.34. The SyntaxWarning printed above it comes from an old line in Ryu's code and does not affect the experiment.

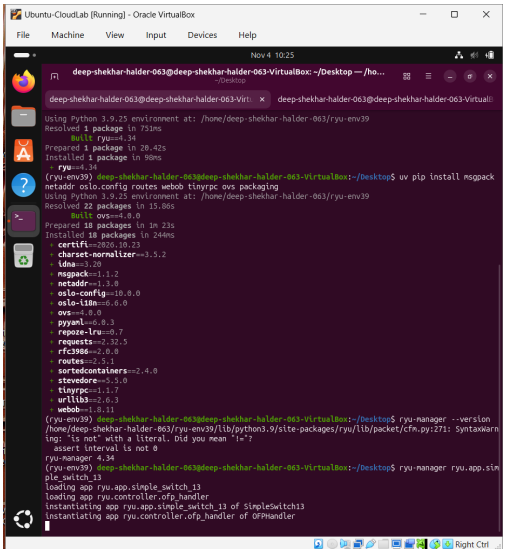


Step 10: Start the Ryu controller

```
ryu-manager ryu.app.simple_switch_13
```

Explanation: Starts Ryu with the simple_switch_13 application, a learning switch that uses OpenFlow 1.3. It listens on port 6653 for switches to connect. This terminal stays open for the rest of the experiment.

Observation: Ryu loaded the application and then waits quietly, which is the expected behaviour until a switch connects.

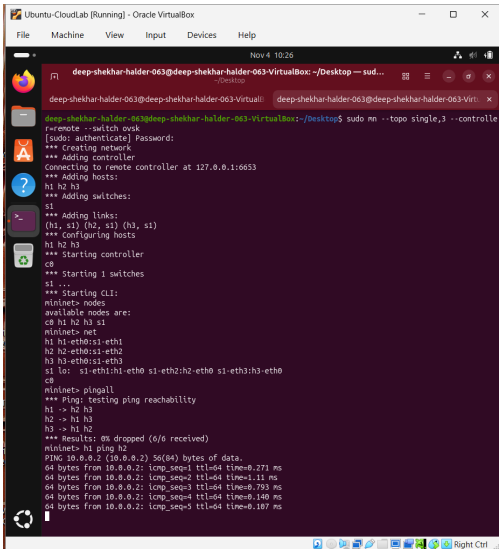


Steps 11 to 14: Create the SDN topology and test connectivity

```
sudo mn --topo single,3 --controller=remote --switch ovsk
mininet> nodes
mininet> net
mininet> pingall
mininet> h1 ping h2
```

Explanation: --topo single,3 creates one switch with three hosts. --controller=remote makes the switch connect to the external Ryu controller (127.0.0.1:6653) instead of Mininet's built-in one. --switch ovsk uses Open vSwitch. nodes lists the devices, net shows the links, and pingall tests every host pair.

Observation: nodes shows c0, h1, h2, h3 and s1. net shows h1, h2 and h3 connected to s1 on ports eth1, eth2 and eth3. pingall reports 0% dropped (6/6 received), and h1 can ping h2 with replies of well under 2 ms.

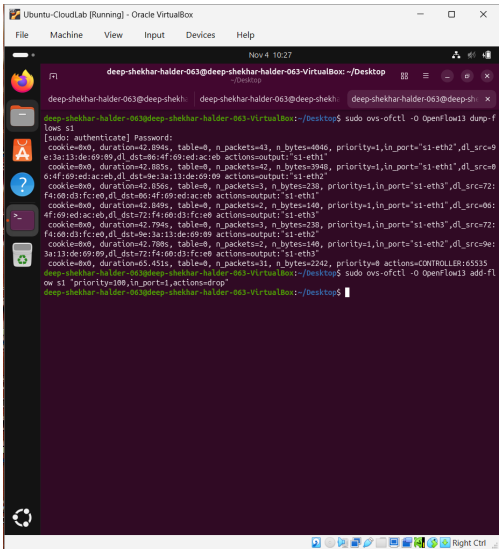


Steps 15 and 16: View flow rules and add a filtering rule

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=1,actions=drop"
```

Explanation: dump-flows lists the flow table of switch s1. The add-flow command inserts a rule that matches every packet arriving on port 1 and drops it. Port 1 is where h1 is connected, and priority 100 is higher than the priority 1 rules learned by Ryu, so it is checked first.

Observation: The flow table contains the rules learned by Ryu (priority 1, matching source and destination MAC addresses and sending packets out of the right port) and a table-miss rule (priority 0) that sends unknown packets to the controller. The drop rule was accepted without error.

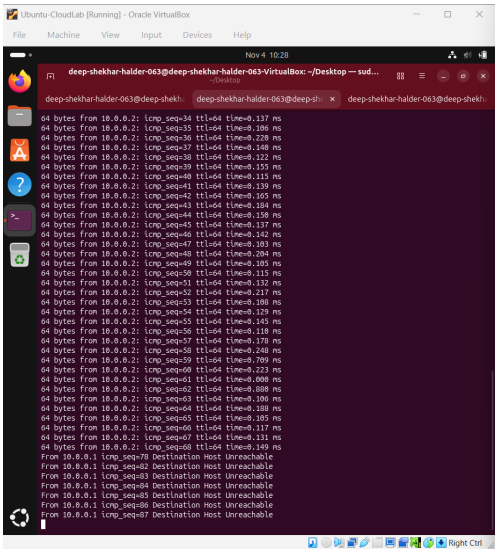


Step 17: Test the filtering rule

```
mininet> h1 ping h2
```

Explanation: Repeats the ping from h1 after the drop rule is installed to see its effect.

Observation: The replies stop and Destination Host Unreachable messages appear. In this run 94 packets were sent and 68 were received before the rule took effect. This shows that the switch drops traffic from h1 as the flow rule instructs.

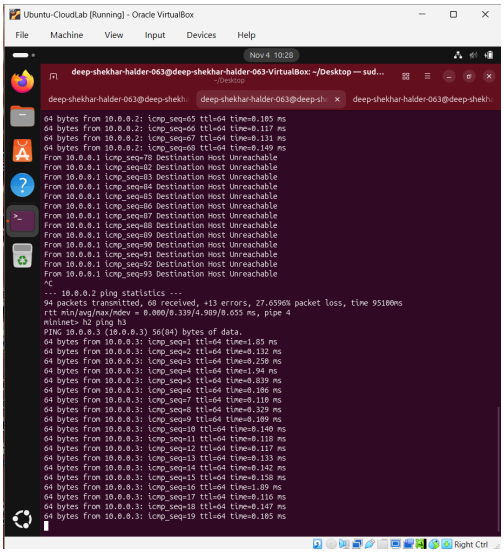


Additional check: other hosts are unaffected

```
mininet> h2 ping h3
```

Explanation: The drop rule only matches port 1, so traffic between h2 and h3 should continue.

Observation: h2 received replies from h3 while the rule was active (19 replies in the screenshot). Later pings in this window failed only because the next step, del-flows, also deleted the flow rules Ryu had installed. That is expected and not an error.

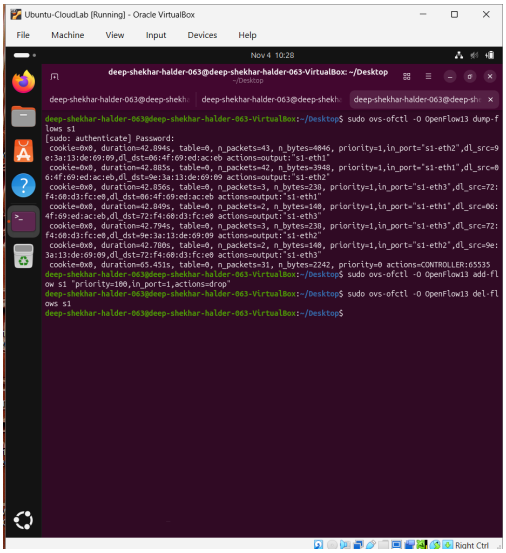


Step 18: Remove flow rules

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

Explanation: Deletes all flow entries from s1, including the manually added drop rule.

Observation: The command ran with no error. Because it removes every entry, the controller's learned rules are removed too, and new ones would be learned again only when new traffic reaches the controller.

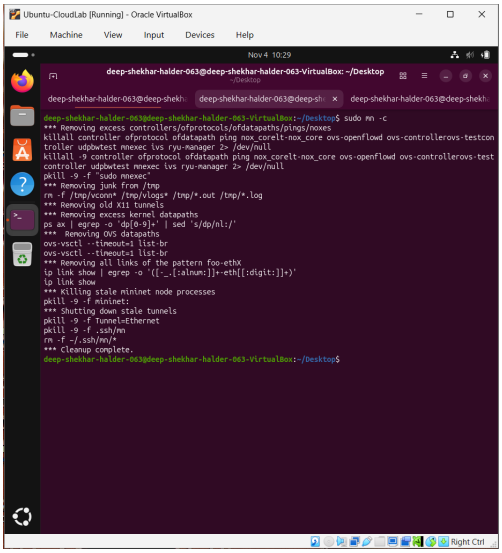


Step 19: Clean Mininet

```
sudo mn -c
```

Explanation: Removes leftover Mininet interfaces, switches and processes so the next experiment starts clean.

Observation: The output ends with Cleanup complete.



Observations Summary

Test	Command	Result
Default Mininet test	sudo mn --test pingall	0% dropped (2/2 received)
SDN topology connectivity	pingall	0% dropped (6/6 received)
Before drop rule	h1 ping h2	Replies received (68 received)
After drop rule on port 1	h1 ping h2	Destination Host Unreachable
After drop rule, other hosts	h2 ping h3	Replies received while the rule was active
Cleanup	sudo mn -c	Cleanup complete

Problems Faced and Fixes

Problem	Fix
apt update returned 403 Forbidden	Changed the apt sources from http:// to https://.
apt install waited for a lock	Ubuntu's automatic updater was running; the install was retried after it finished.
pip install ryu failed on the new Python	Created a Python 3.9 environment with uv and installed Ryu with older setuptools and eventlet.
VM crashed with a kernel panic at start	Set the VirtualBox paravirtualization interface to None and restarted the VM.

Conclusion

The experiment showed how SDN separates control from forwarding. Ryu, the controller, learned the hosts and programmed Open vSwitch with forwarding rules, giving full connectivity between h1, h2 and h3. A single manually added OpenFlow rule with a higher priority was enough to block one host without affecting the others, and deleting the rules returned the switch to an empty flow table. This demonstrates the central control and programmability that SDN offers.